Les vues en Django

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les vues basées sur des fonctions **3**

A. Création d'une première vue 3

B. Accéder aux données du modèle 4

C. Gestion des erreurs 4

D. Exercice : Quiz 4

III. Mise en forme avec les gabarits (templates) **5**

A. Création de templates 5

B. Insérer de la logique dans les templates 5

C. Exercice : Quiz 6

IV. Les vues basées sur des classes **7**

A. Les vues basées sur des classes 7

B. Exercice : Quiz 8

V. Aller plus loin **9**

A. Héritage et inclusion de templates 9

B. Gestion des fichiers statiques 9

C. Formulaires et redirections 9

D. Exercice : Quiz 11

VI. L'essentiel **12**

VII. Auto-évaluation **12**

A. Exercice 12

B. Test 13

Solutions des exercices **14**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 75 minutes

Environnement de travail : Python 3.10+, Django 4.2+, un éditeur de texte (PyCharm, VS Code, etc.), un navigateur web.

Pré requis : Bases en Python, notions de base du développement web (HTML, CSS), connaissances des principes de base Django.

Contexte

Dans ce cours, nous allons nous focaliser sur les vues de Django, des composants clés pour la gestion de la logique de votre application. La création, la gestion des erreurs et la mise en forme des réponses retournées par les vues constituent des compétences indispensables à développer pour travailler efficacement avec Django. Nous allons vous guider à travers ces démarches, vous offrant une compréhension approfondie qui vous permettra d'interagir avec le système de requêtes-réponses de manière efficiente et sécurisée.

II. Les vues basées sur des fonctions

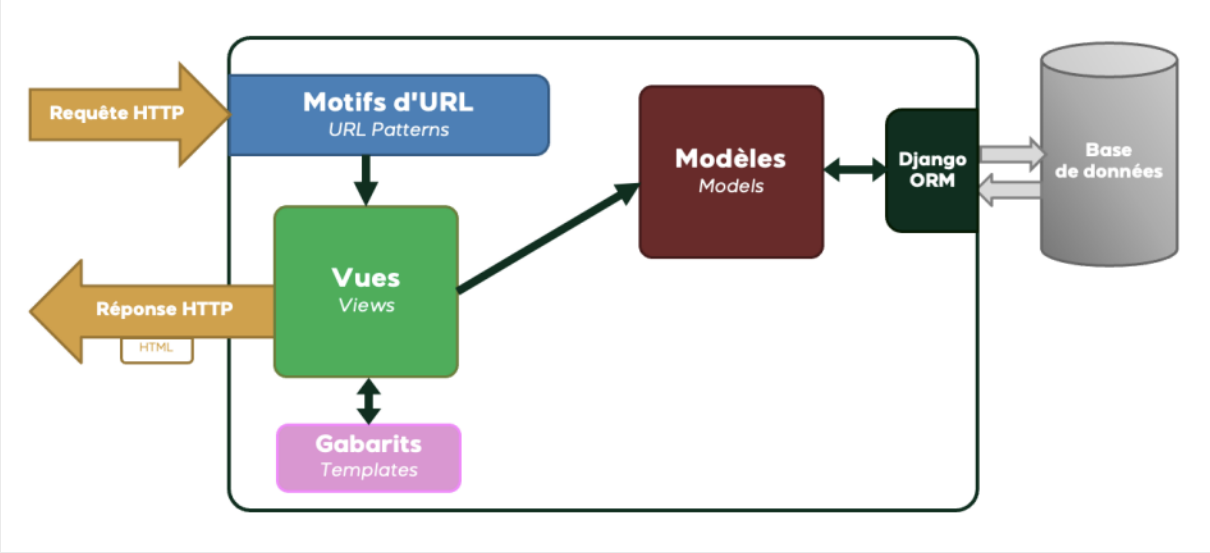
A. Création d'une première vue

Rappel **Concept de vues en Django**

Django utilise le concept de « *vues* » pour gérer la logique de votre application. Les vues, définies comme des fonctions ou des classes Python, servent à traiter les requêtes HTTP et à produire les réponses HTTP adéquates. Les vues Django agissent comme des ponts entre vos modèles et vos templates, prenant des données de vos modèles, les traitant si nécessaire, et les passant à un template pour la présentation.

Un des avantages des vues Django réside dans leur flexibilité et leur modularité. Grâce à leur conception fonctionnelle ou orientée objet, les vues peuvent être facilement réutilisées et étendues, favorisant un code propre et maintenable. Avec les vues Django, les tâches courantes comme le traitement des formulaires, l'affichage des listes d'objets, ou la manipulation des sessions deviennent des opérations simples et structurées.

De plus, les vues Django offrent un niveau élevé d'intégration avec le reste du framework. Elles interagissent de manière transparente avec le système de routage d'URL de Django, permettant une correspondance flexible et puissante entre les URL et les vues. Les vues peuvent également tirer parti des fonctionnalités de sécurité de Django, telles que la protection contre les attaques CSRF ou l'authentification des utilisateurs.



La définition des vues se fait au niveau du fichier *views.py*.

Dans le cas d'une vue basée sur une fonction, celle-ci doit prendre un objet HttpRequest en paramètre, et retourner un objet de type HttpResponse.Il ne faut pas oublier d'enregistrer la vue dans le fichier d'URLs de l'application de la manière suivante :

```
1 from django.urls import path
2 from .views import first_view
3
4 urlpatterns = [
5     path('hello', first_view),
6 ]
```

B. Accéder aux données du modèle

Les vues communiquent avec les modèles via les QuerySets.

Dans le cas où une donnée, comme un ID par exemple, doit être transmise par l'utilisateur via l'URL, il faut indiquer une variable d'URL dans le pattern utilisé dans la fonction path(), en précisent le nom de celle-ci, ainsi que son type.

La valeur de cette variable d'URL sera accessible via les paramètres de la fonction de la vue associée à cette URL.

C. Gestion des erreurs

Une erreur non gérée peut conduire à une page d'erreur inattendue pour l'utilisateur, et peut même causer des problèmes de sécurité. Django offre plusieurs mécanismes pour gérer les erreurs dans les vues.

L'une des erreurs les plus courantes est le non-respect des contraintes de la base de données, comme tenter de récupérer un enregistrement qui n'existe pas. Par exemple, dans la vue un_seul_etudiant, si aucun étudiant ne correspond à l'ID fourni, l'ORM Django lèvera une exception Etudiant.DoesNotExist. Nous devons gérer cette exception et fournir une réponse appropriée.

Nous utilisons la fonction get_object_or_404 qui essaie de récupérer l'étudiant correspondant à l'ID fourni. Si aucun étudiant ne correspond, elle renvoie automatiquement une réponse HTTP 404, indiquant que la ressource demandée n'a pas été trouvée.

C'est un exemple de gestion d'erreur simple, mais efficace. La fonction get_object_or_404 simplifie le code de la vue, tout en fournissant une bonne expérience utilisateur en cas d'erreur.

```
1 def un_seul_etudiant(request: HttpRequest, pk: int):
2     etudiant = get_object_or_404(Etudiant, pk=pk)
3     context = {'etudiant': etudiant}
4     return render(request, 'myapp/etudiant.html', context)
```

D. Exercice : Quiz

[solution n°1 p.15]

Question 1

Qu'est-ce qu'une vue dans le contexte de Django ?

- ○ Une partie du système de routage d'URL
- ○ Une fonction ou une classe Python servant à traiter les requêtes HTTP et à produire les réponses HTTP
- ○ Un système de gestion de base de données

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quelle est l'une des forces principales des vues Django ?

- Leur flexibilité et leur modularité
- Leur capacité à stocker des données
- Leur capacité à interagir avec le système de fichiers

Question 3

Comment créer une vue dans Django ?

- En définissant un modèle
- En définissant une fonction ou une classe
- En définissant un template

Question 4

Quelle exception est levée par l'ORM de Django lorsqu'un enregistrement demandé n'existe pas dans la base de données ?

- ValueError
- NotFoundError
- ObjectDoesNotExist

Question 5

Quelle fonction de Django permet de simplifier la gestion des erreurs lors de la récupération d'objets à partir de la base de données ?

- get_object_or_fail()
- get_object_or_404()
- get_object_or_none()

III. Mise en forme avec les gabarits (templates)

A. Création de templates

Django définit son propre langage de template, basé sur la syntaxe HTML.

Il s'agit en réalité de fichiers HTML enrichis par des balises Django, sous la forme suivante :

```
{{ valeur }}
```

Les balises Django permettent notamment d'accéder aux données transmises par la vue au template.

B. Insérer de la logique dans les templates

Tous droits réservés à STUDI - Reproduction interdite

La syntaxe des templates de Django permet d'insérer de la logique dans les templates, par exemple conditionner l'affichage de certaines informations dans la page, ou bien des structures de boucles qui permettent par exemple de répéter N fois un certain bloc dans la page. Cela est très utile, notamment pour mettre en forme les listes ou les tableaux.

Contrairement aux balises de template qui permettent d'afficher de la donnée, la syntaxe des balises logiques de Django est la suivante :

```
1 {% nom_de_la_balise %}
```

C. Exercice : Quiz

[solution n°2 p.16]

Question 1

Comment accède-t-on aux données transmises par la vue dans un template Django ?

- Utiliser des balises HTML standard
- Utiliser des balises de la forme {{ valeur }}
- Utiliser des balises de la forme {% valeur %}
- Utiliser des fonctions Python dans le template

Question 2

À quoi sert la fonction render() dans Django ?

- À envoyer une réponse HTTP 404
- À générer une réponse HTTP en combinant un template avec un contexte
- À rediriger vers une autre vue
- À récupérer les données du modèle dans la vue

Question 3

Quelle syntaxe est utilisée dans un template Django pour insérer de la logique conditionnelle ?

- {% if condition %} ... {% endif %}
- {{ if condition }} ... {{ endif }}
- if (condition) { … }
- <if condition> ... </endif>

Question 4

Quel est l'avantage d'utiliser des boucles dans les templates Django ?

- Réduire le nombre de requêtes SQL
- Répéter du code HTML plusieurs fois pour différents éléments
- Inclure des fragments de templates
- Créer des animations interactives

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Que se passe-t-il si vous tentez d'accéder à une variable inexistante dans un template Django ?

- Une erreur 404 est générée
- Le serveur Django plante
- La variable est affichée comme une chaîne vide
- La variable est affichée telle quelle, entre double accolades

Question 6

Comment transmet-on un contexte vide à la fonction render() ?

- En passant None en tant que troisième argument
- En passant un dictionnaire vide en tant que troisième argument
- En omettant complètement le troisième argument
- Les trois réponses sont possibles

IV. Les vues basées sur des classes

A. Les vues basées sur des classes

La plupart des vues basées sur des classes dérivent de la classe View intégrée dans Django, mais Django fournit également un certain nombre de vues basées sur des classes génériques qui implémentent des fonctionnalités courantes. Par exemple, Django fournit une vue générique pour afficher une liste d'objets ou une vue générique pour afficher un détail d'objet.

Exemple de vue générique afin d'afficher une liste de professeurs :

```
1 from django.views.generic.list import ListView
2 from .models import Professeur
3
4 class ListProfesseurs(ListView):
5     model = Professeur
```

Le template permettant de formater l'affichage de cette vue :

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>Tous les professeurs</title>
6 </head>
7 <body>
8     <h1>Tous les professeurs</h1>
9     <ul>
10         {% for professeur in professeur_list %}
11             <li>
12                 {{ professeur.prenom }} {{ professeur.nom }}
13             </li>
14         {% endfor %}
15     </ul>
16 </body>
17 </html>
```

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°3 p.17]

Question 1

Quelle est l'avantage principal des vues basées sur des classes (CBV) par rapport aux vues basées sur des fonctions (FBV) ?

- Elles sont plus rapides et plus performantes
- Elles permettent une meilleure réutilisation et organisation du code
- Elles consomment moins de mémoire
- Elles ne nécessitent pas d'importer de modules supplémentaires

Question 2

Quelle classe Django est souvent utilisée comme base pour créer une vue affichant une liste d'objets ?

- TemplateView
- DetailView
- ListView
- FormView

Question 3

Quel est le nom de l'attribut utilisé dans les vues basées sur des classes pour définir le template associe à celle-ci ?

- template
- template_class
- template_view
- template_name

Question 4

Dans le fichier d'urls, quelle méthode faut-il invoquer sur une vue basée sur une classe afin de l'associer à un pattern d'URL ?

- to_view()
- is_view()
- get_view()
- as_view()

Question 5

Dans une vue basée sur une classe, à quoi sert la méthode get() ?

- Traiter les requêtes HTTP de type GET
- Récupérer une instance du modèle
- Sélectionner le template adéquat

Tous droits réservés à STUDI - Reproduction interdite

V. Aller plus loin

A. Héritage et inclusion de templates

La syntaxe des templates Django fournit des balises permettant la factorisation du code HTML.

La balise {% extends %} permet, selon le même principe que celui de la programmation orientée objet, de mettre en place un héritage de template ; ceci afin de factoriser le code commun à travers différents templates.

La balise {% includes %} permet d'externaliser certaines portions de template, et de les réutiliser potentiellement à plusieurs endroits différents.

B. Gestion des fichiers statiques

La gestion des static files avec Django

En développement web, les « *static files* » se réfèrent à tous les fichiers qui ne sont pas générés dynamiquement. Cela inclut des fichiers tels que les feuilles de style CSS, les scripts JavaScript, les images, les fichiers audio, etc. Ces éléments sont essentiels pour donner vie à vos pages web, en ajoutant des styles, des fonctionnalités et des médias.

Django fournit une infrastructure robuste pour gérer ces fichiers statiques. Pour chaque application que vous créez dans Django, un dossier appelé « *static* » est généralement créé. C'est ici que vous stockerez tous les fichiers qui ne changent pas et qui sont nécessaires à votre application, tels que les images, les feuilles de style CSS et les scripts JavaScript. Lorsque vous exécutez votre application en mode développement, Django est capable de trouver et de servir ces fichiers automatiquement, ce qui simplifie grandement le processus de développement.

La structuration des static files

La structuration de vos fichiers statiques dans chaque application peut être organisée comme vous le souhaitez, mais il est courant de créer des sous-répertoires pour différents types de fichiers, comme « *css* » pour les feuilles de style, « *js* » pour les scripts JavaScript et « *img* » pour les images.

Attention **Les static files en production**

Lorsque votre application passe en mode production, la gestion des fichiers statiques est différente. Django n'est pas conçu pour servir des fichiers statiques en production et il est généralement recommandé de configurer un serveur web séparé, ou un service de stockage de fichiers statiques, pour servir ces fichiers. Des services comme Nginx, Apache, ou même des services de stockage en nuage comme AWS S3, sont souvent utilisés à cet effet.

Remarque **Les fichiers statiques et la performance**

Le stockage et le service efficaces des fichiers statiques sont essentiels pour la performance de votre application. En optimisant la façon dont ces fichiers sont servis, vous pouvez améliorer le temps de chargement de votre site, ce qui peut avoir un impact positif sur l'expérience de l'utilisateur et le référencement de votre site.

C. Formulaires et redirections

Redirections

Dans le cadre du développement web, il est souvent nécessaire de rediriger les utilisateurs vers une autre page après qu'ils aient effectué une certaine action, comme soumettre un formulaire ou effectuer une action CRUD (Create, Read, Update, Delete). Les redirections sont cruciales pour maintenir une expérience utilisateur fluide et cohérente, et Django fournit une fonction très pratique pour gérer ces redirections : la fonction redirect().

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Gérer les redirections**

Dans Django, la fonction redirect() est un raccourci pour créer une réponse HTTP avec le statut 302, qui indique une redirection temporaire. Cette fonction est très flexible, car elle peut prendre plusieurs types d'arguments pour déterminer l'URL de redirection.

Par exemple, vous pouvez passer directement l'URL en tant que chaîne de caractères ou vous pouvez passer le nom d'une vue et ses arguments correspondants et Django générera l'URL pour vous. Cela vous donne une grande flexibilité pour gérer les redirections dans votre application, et vous assure que vos utilisateurs seront toujours dirigés vers l'endroit approprié.

```
1 from django.shortcuts import render, redirect
2 from .forms import PostForm
3
4 def create_post(request):
5     if request.method == "POST":
6         form = PostForm(request.POST)
7         if form.is_valid():
8             form.save()
9             return redirect('home')
10        else:
11            form = PostForm()
12            return render(request, 'app/create_post.html', {'form', form})
```

Ici, create_post est une vue qui gère la requête POST pour créer un nouveau Post. Si la requête est une méthode POST et que le formulaire est valide, la vue sauvegarde le formulaire puis redirige l'utilisateur vers l'URL mappée à 'home'.

Si la requête n'est pas une méthode POST (par exemple, si l'utilisateur accède à la page pour la première fois), la vue crée une nouvelle instance de PostForm et la passe template 'app/create_post.html'.

Méthode **Gestion des erreurs dans les vues et les redirections**

Lors de la gestion des vues et des redirections, il faut vous assurer une bonne gestion des erreurs. Par exemple, si un formulaire n'est pas valide, il faut s'assurer que l'utilisateur reçoit un retour approprié. De même, lors de la redirection, il est important de vérifier que l'URL de redirection est correcte et accessible. Sans une gestion adéquate des erreurs, votre application peut se comporter de manière imprévisible et frustrer les utilisateurs.

Méthode **Gestion des erreurs dans les vues et les redirections**

L'une des étapes de toute application web basée sur Django est la validation de formulaire et la gestion d'erreurs. Chaque formulaire soumis à Django passe par un processus de validation, lors duquel Django vérifie chaque champ pour s'assurer qu'il répond à toutes les contraintes et conditions définies. Si le formulaire n'est pas valide, Django associe des messages d'erreur pertinents à chaque champ fautif. L'attribut errors du formulaire offre un accès direct à ces messages d'erreur, fournissant un dictionnaire où chaque champ est associé à sa liste d'erreurs. L'intégration de ces erreurs dans le template de réponse peut se faire simplement en faisant référence à

{{ form.errors }} ou {{ form.nom_du_champ.errors }}.

Un exemple de l'affichage des erreurs d'un champ de formulaire dans un template est donné ci-dessous :

```
1 {% if form.nom_du_champ.errors %}
2     <ul>
3     {% for error in form.nom_du_champ.errors %}
4         <li>{{ error }}</li>
5     {% endfor %}
6     </ul>
7 {% endif %}
```


Tous droits réservés à STUDI - Reproduction interdite

Méthode **Gestion des erreurs de redirection**

Dans une application Django, les redirections sont souvent utilisées pour orienter le flux de l'application et guider l'utilisateur à travers différents points de vue et fonctionnalités. Cependant, les redirections peuvent elles-mêmes être source d'erreurs si l'URL cible n'existe pas ou est inaccessible pour une raison quelconque. Pour gérer de tels scénarios, Django fournit l'exception Http404. Lorsque levée, cette exception entraîne l'affichage d'une page d'erreur 404, informant l'utilisateur que l'URL demandée n'a pas été trouvée.

Voici un exemple de code illustrant la gestion d'une erreur de redirection en utilisant l'exception Http404 :

```
1 from django.shortcuts import redirect, get_object_or_404
2 from .models import Post
3
4 def view_post(request, post_id):
5     post = get_object_or_404(Post, id=post_id)
6     return render(request, 'app/view_post.html', {'post': post})
7
8 def delete_post(request, post_id):
9     post = get_object_or_404(Post, id=post_id)
10    post.delete()
11    return redirect('home')
```

Dans cet exemple, view_post et delete_post sont des vues qui gèrent respectivement l'affichage et la suppression d'un Post. Si le Post spécifié par post_id n'existe pas, get_object_or_404 lève une exception Http404, et Django affiche une page d'erreur 404.

En résumé, la gestion efficace des vues, du routage et des redirections dans Django est un élément essentiel pour assurer une expérience utilisateur fluide. De plus, la manipulation des formulaires et la gestion adéquate des erreurs sont des composantes clés de cette gestion.

D. Exercice : Quiz

[solution n°4 p.18]

Question 1

Quelle balise Django est utilisée pour indiquer qu'un template hérite d'un autre template ?

- {% inherits %}
- {% extends %}
- {% block %}
- {% super %}

Question 2

Comment charge-t-on un fichier CSS statique dans un template Django ?

- <link rel="stylesheet" href="{% load static %}/styles.css">
- <link rel="stylesheet" href="{% static 'styles.css' %}">
- <link rel="stylesheet" href="{% url 'styles.css' %}">
- <link rel="stylesheet" href="{% include 'styles.css' %}">

Question 3

Tous droits réservés à STUDI - Reproduction interdite

À quoi sert la fonction redirect dans Django ?

- À créer une réponse HTTP avec le statut 302, qui indique une redirection temporaire
- À afficher des messages d'erreur
- À créer des formulaires

Question 4

Dans un formulaire, quelle méthode est utilisée pour vérifier si celui-ci est valide ?

- is_valid()
- check()
- validate()
- clean()

Question 5

Comment spécifie-t-on un lien vers une autre vue à l'intérieur d'un template ?

- {% link 'name' %}
- {% url 'name' %}
- {% href 'name' %}
- {% redirect 'name' %}

VI. L'essentiel

Dans l'exploration de Django, il est crucial de maîtriser les principes fondamentaux liés à la création de vues, à la gestion des erreurs et à la redirection. Les vues, qui sont au cœur de toute application Django, représentent les fonctions Python qui prennent une requête web et produisent une réponse web. Savoir comment construire et manipuler ces vues est capital pour gérer le flux d'informations dans votre application.

Que ce soit la définition d'une vue simple renvoyant un message « *Hello, World!* », la gestion des erreurs en levant, par exemple, une erreur 404 si l'utilisateur n'est pas authentifié, chaque action a une importance cruciale pour le fonctionnement de votre application. En plus de cela, comprendre le système de routage de Django, qui permet de mapper les vues aux URL, est fondamental.

Que ce soit l'élaboration d'une règle de routage mappant toute requête à l'URL '/hello/' à notre vue hello_world, ou l'utilisation de la fonction de redirection pour renvoyer l'utilisateur à une autre URL, chaque étape doit être soigneusement réalisée pour assurer une navigation fluide à l'utilisateur. En somme, la maîtrise des vues, du système de routage et de la gestion des erreurs dans Django est une compétence indispensable pour tout développeur Django.

VII. Auto-évaluation

A. Exercice

Question 1

[solution n°5 p.20]

Écrivez le code pour une vue all_posts qui récupère tous les posts de votre base de données et les envoie à un template appelé all_posts.html.

Question 2

[solution n°6 p.20]

Maintenant, écrivez le code pour une vue post_detail qui récupère un post spécifique en utilisant son id et l'envoie à un template appelé post_detail.html.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

[solution n°7 p.20]

Enfin, reliez ces vues aux URL appropriées dans votre fichier urls.py.

B. Test

Exercice 1 : Quiz

[solution n°8 p.20]

Question 1

Quelle balise est utilisée pour définir une section modifiable dans un template parent ?

- {% define %}
- {% block %}
- {% content %}
- {% section %}

Question 2

Quel fichier Django contient les configurations pour la gestion des templates (notamment le chemin du répertoire de base de ceux-ci) ?

- urls.py
- views.py
- settings.py
- admin.py

Question 3

Quelle instance est retournée pour créer une réponse HTTP dans une vue basée sur une fonction ?

- HttpResponse()
- HttpRender()
- TemplateRender()
- TemplateView()

Question 4

Quelle est la syntaxe correcte pour inclure un fichier JavaScript dans un template Django ?

- <script src="{% load static %}/script.js"></script>
- <script src="{% static 'script.js' %}"></script>
- <script src="{% url 'script.js' %}"></script>
- <script src="{% import 'script.js' %}"></script>

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Dans quel dossier Django recommande-t-il de stocker les templates ?

- ○ Dans un répertoire nommé templates
- ○ Dans le répertoire views
- ○ À la racine de l'application
- ○ Cela n'a pas d'importance

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 4 Solution n°1

Question 1

Qu'est-ce qu'une vue dans le contexte de Django ?

- ○ Une partie du système de routage d'URL
- ◉ Une fonction ou une classe Python servant à traiter les requêtes HTTP et à produire les réponses HTTP
- ○ Un système de gestion de base de données

En Django, une vue est représentée par une fonction ou une classe Python qui est chargée de gérer les interactions entre les requêtes HTTP et les réponses correspondantes. Elle fait le lien entre les modèles (représentant les données) et les templates (représentant la présentation).

Question 2

Quelle est l'une des forces principales des vues Django ?

- ◉ Leur flexibilité et leur modularité
- ○ Leur capacité à stocker des données
- ○ Leur capacité à interagir avec le système de fichiers

Une des grandes qualités des vues Django est leur adaptabilité et leur capacité à être modulaires. Elles sont conçues de manière à favoriser un code organisé et facile à maintenir, qu'elles soient basées sur des fonctions ou des classes.

Question 3

Comment créer une vue dans Django ?

- ○ En définissant un modèle
- ◉ En définissant une fonction ou une classe
- ○ En définissant un template

Pour instaurer une vue dans Django, vous devez définir une fonction ou une classe qui interagira avec les requêtes HTTP pour fournir une réponse appropriée. Cette réponse est souvent le résultat de l'interaction entre les modèles (données) et les templates (présentation).

Question 4

Quelle exception est levée par l'ORM de Django lorsqu'un enregistrement demandé n'existe pas dans la base de données ?

- ○ ValueError
- ○ NotFoundError
- ◉ ObjectDoesNotExist

Lorsque vous tentez d'accéder à un enregistrement inexistant dans la base de données à travers l'ORM de Django, l'exception ObjectDoesNotExist est générée. Il est nécessaire de prévoir un mécanisme de gestion pour cette exception afin de fournir une réponse adéquate.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Quelle fonction de Django permet de simplifier la gestion des erreurs lors de la récupération d'objets à partir de la base de données ?

- ○ get_object_or_fail()
- ◉ get_object_or_404()
- ○ get_object_or_none()

Q Django propose la fonction get_object_or_404 qui tente de récupérer un objet spécifique dans la base de données. Si l'objet n'est pas trouvé, cette fonction renvoie directement une réponse HTTP 404, simplifiant ainsi la gestion des erreurs.

Exercice p. 6 Solution n°2

Question 1

Comment accède-t-on aux données transmises par la vue dans un template Django ?

- ○ Utiliser des balises HTML standard
- ◉ Utiliser des balises de la forme {{ valeur }}
- ○ Utiliser des balises de la forme {% valeur %}
- ○ Utiliser des fonctions Python dans le template

Q Les données provenant de la vue sont affichées à l'aide des doubles accolades.

Question 2

À quoi sert la fonction render() dans Django ?

- ○ À envoyer une réponse HTTP 404
- ◉ À générer une réponse HTTP en combinant un template avec un contexte
- ○ À rediriger vers une autre vue
- ○ À récupérer les données du modèle dans la vue

Q La fonction render() est l'un des raccourcis fournis par Django. Elle transmet les données provenant dans la vue au template, et met en forme la réponse HTTP en combinant les deux.

Question 3

Quelle syntaxe est utilisée dans un template Django pour insérer de la logique conditionnelle ?

- ◉ {% if condition %} ... {% endif %}
- ○ {{ if condition }} ... {{ endif }}
- ○ if (condition) { … }
- ○ <if condition> ... </endif>

Q Cela est valable également pour les boucles.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quel est l'avantage d'utiliser des boucles dans les templates Django ?

- ○ Réduire le nombre de requêtes SQL
- ◉ Répéter du code HTML plusieurs fois pour différents éléments
- ○ Inclure des fragments de templates
- ○ Créer des animations interactives

Il s'agit d'un principe similaire à celui des boucles classiques en Python.

Question 5

Que se passe-t-il si vous tentez d'accéder à une variable inexistante dans un template Django ?

- ○ Une erreur 404 est générée
- ○ Le serveur Django plante
- ◉ La variable est affichée comme une chaîne vide
- ○ La variable est affichée telle quelle, entre double accolades

Il faut donc veiller à toujours faire référence à des clés existantes dans le dictionnaire de contexte, car aucun message d'erreur n'est affiché, y compris en mode Debug .

Question 6

Comment transmet-on un contexte vide à la fonction render() ?

- ○ En passant None en tant que troisième argument
- ○ En passant un dictionnaire vide en tant que troisième argument
- ○ En omettant complètement le troisième argument
- ◉ Les trois réponses sont possibles

Le troisième argument de la fonction render() attend un dictionnaire, cependant il possède une valeur par défaut « *None* », le rendant facultatif. Les trois réponses sont donc correctes.

Exercice p. 8 Solution n°3

Question 1

Quelle est l'avantage principal des vues basées sur des classes (CBV) par rapport aux vues basées sur des fonctions (FBV) ?

- ○ Elles sont plus rapides et plus performantes
- ◉ Elles permettent une meilleure réutilisation et organisation du code
- ○ Elles consomment moins de mémoire
- ○ Elles ne nécessitent pas d'importer de modules supplémentaires

L'utilisation des vues génériques et des mixins permet une meilleure réutilisation du code et une écriture simplifiée.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Quelle classe Django est souvent utilisée comme base pour créer une vue affichant une liste d'objets ?

- ○ TemplateView
- ○ DetailView
- ◉ ListView
- ○ FormView

Parmi les vues génériques, ListView fournit un mécanisme qui récupère automatiquement toutes les instances en base de données du modèle associé à la vue, et les transmet automatiquement au template associé sous la forme d'une liste.

Question 3

Quel est le nom de l'attribut utilisé dans les vues basées sur des classes pour définir le template associe à celle-ci ?

- ○ template
- ○ template_class
- ○ template_view
- ◉ template_name

Cela permet de définir ou de personnaliser le nom du fichier de template associé à la vue.

Question 4

Dans le fichier d'urls, quelle méthode faut-il invoquer sur une vue basée sur une classe afin de l'associer à un pattern d'URL ?

- ○ to_view()
- ○ is_view()
- ○ get_view()
- ◉ as_view()

L'appel à as_view() est nécessaire pour les vues basées sur des classes.

Question 5

Dans une vue basée sur une classe, à quoi sert la méthode get() ?

- ◉ Traiter les requêtes HTTP de type GET
- ○ Récupérer une instance du modèle
- ○ Sélectionner le template adéquat

Il existe également la méthode post() permettant de traiter notamment les formulaires.

Exercice p. 11 **Solution n°4**

Tous droits réservés à STUDI - Reproduction interdite

studi

Question 1

Quelle balise Django est utilisée pour indiquer qu'un template hérite d'un autre template ?

- ○ {% inherits %}
- ◉ {% extends %}
- ○ {% block %}
- ○ {% super %}

L'héritage de template avec {% extends %} permet la factorisation de templates, et l'injection de fragments de template dans certaines sections de template enfant à l'aide de {% block %}.

Question 2

Comment charge-t-on un fichier CSS statique dans un template Django ?

- ○ <link rel="stylesheet" href="{% load static %}/styles.css">
- ◉ <link rel="stylesheet" href="{% static 'styles.css' %}">
- ○ <link rel="stylesheet" href="{% url 'styles.css' %}">
- ○ <link rel="stylesheet" href="{% include 'styles.css' %}">

L'utilisation de ressources statiques se fait grâce à la balise {% static %}.

Question 3

À quoi sert la fonction redirect dans Django ?

- ◉ À créer une réponse HTTP avec le statut 302, qui indique une redirection temporaire
- ○ À afficher des messages d'erreur
- ○ À créer des formulaires

Il s'agit d'une fonction raccourci qui permet d'effectuer une redirection (code HTTP 302).

Question 4

Dans un formulaire, quelle méthode est utilisée pour vérifier si celui-ci est valide ?

- ◉ is_valid()
- ○ check()
- ○ validate()
- ○ clean()

Cette fonction vérifie si les données saisies dans le formulaire sont conformes aux exigences des validateurs, et retourne un booléen.

Question 5

Comment spécifie-t-on un lien vers une autre vue à l'intérieur d'un template ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ {% link 'name' %}
- ◉ {% url 'name' %}
- ○ {% href 'name' %}
- ○ {% redirect 'name' %}

La balise {% url %} crée un lien dynamiquement vers une vue, à partir de son nom (tel que défini dans le fichier d'URLs).

p. 12 Solution n°5

```
1 from django.shortcuts import render
2 from .models import Post
3
4 def all_posts(request):
5     posts = Post.objects.all()
6     return render(request, 'all_posts.html', {'posts': posts})
```

Ici, nous utilisons la méthode objects.all() de Django pour récupérer tous les posts de notre base de données. Nous les passons ensuite au template all_posts.html à travers le contexte.

p. 12 Solution n°6

```
1 from django.shortcuts import render, get_object_or_404
2 from .models import Post
3
4 def post_detail(request, id):
5     post = get_object_or_404(Post, id=id)
6     return render(request, 'post_detail.html', {'post': post})
```

Ici, nous utilisons la fonction get_object_or_404 de Django pour récupérer le post spécifique ou renvoyer une page d'erreur 404 si le post n'existe pas. Nous passons ensuite le post au template post_detail.html.

p. 13 Solution n°7

```
1 from django.urls import path
2 from .views import all_posts, post_detail
3
4 urlpatterns = [
5     path('posts/', all_posts, name='all_posts'),
6     path('posts/<int:id>/', post_detail, name='post_detail'),
7 ]
```

Ici, nous utilisons la fonction path de Django pour définir nos URL. La vue all_posts est accessible via l'URL '/posts/', et la vue post_detail est accessible via l'URL '/posts/<id>/', où <id> est l'identifiant du post spécifique.

Exercice p. 13 Solution n°8

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Quelle balise est utilisée pour définir une section modifiable dans un template parent ?

- ○ {% define %}
- ◉ {% block %}
- ○ {% content %}
- ○ {% section %}

L'héritage de template avec {% extends %} permet la factorisation de templates, et l'injection de fragments de template dans certaines sections de template enfant à l'aide de {% block %}.

Question 2

Quel fichier Django contient les configurations pour la gestion des templates (notamment le chemin du répertoire de base de ceux-ci) ?

- ○ urls.py
- ○ views.py
- ◉ settings.py
- ○ admin.py

Le fichier settings.py centralise les configurations du projet Django, notamment celles relatives aux templates.

Question 3

Quelle instance est retournée pour créer une réponse HTTP dans une vue basée sur une fonction ?

- ◉ HttpResponse()
- ○ HttpRender()
- ○ TemplateRender()
- ○ TemplateView()

Les réponses HTTP sont construites à partir d'objets de type HttpResponse.

Question 4

Quelle est la syntaxe correcte pour inclure un fichier JavaScript dans un template Django ?

- ○ <script src="{% load static %}/script.js"></script>
- ◉ <script src="{% static 'script.js' %}"></script>
- ○ <script src="{% url 'script.js' %}"></script>
- ○ <script src="{% import 'script.js' %}"></script>

L'utilisation de ressources statiques se fait grâce à la balise {% static %}.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Dans quel dossier Django recommande-t-il de stocker les templates ?

- ⦿ Dans un répertoire nommé templates
- ○ Dans le répertoire views
- ○ À la racine de l'application
- ○ Cela n'a pas d'importance

Il est possible de modifier cela dans le fichier settings.py. Il est également recommandé de créer un sous-dossier dans « *templates* », portant le même nom que l'application.

Tous droits réservés à STUDI - Reproduction interdite